
Performance-Driven Policy Optimization for Speculative Decoding with Adaptive Windowing

Jie Jiang Xing Sun Ruotian Chen Jianan Su Kaixin Shen

Abstract

Speculative decoding accelerates LLM inference by having a lightweight draft model propose speculative windows of candidate tokens for parallel verification by a larger target model. In practice, speculative efficiency is often bottlenecked by hard-to-draft positions, where an early mismatch truncates the accepted prefix and invalidates the rest of the speculative window. Most learning-based drafters are still optimized with token-level supervised objectives, even though speculative utility is inherently window-level and prefix-sensitive. We propose **PPOW** (Performance-Driven Policy Optimization with Adaptive Windowing), a reinforcement learning framework that shifts drafter optimization from token-level imitation to window-level optimization. PPOW combines a Cost-Aware Speedup Reward, a Distribution-Based Proximity Reward, and Adaptive Divergence-Aware Windowing, which prioritizes informative windows with high confidence-weighted draft-target divergence. PPOW achieves average acceptance lengths of 6.29–6.52 and speedups of $3.39\text{--}4.36\times$ across multiple model families and benchmarks under a unified decoding protocol. These results show that performance-driven window-level optimization is a practical approach to improving speculative decoding efficiency.

1 Introduction

Speculative decoding [1, 2] accelerates Large Language Model (LLM) generation while preserving the output distribution of the target model. At each speculative step, a lightweight draft model (drafter) proposes a *speculative window* of candidate tokens, which the target model then verifies in parallel. In practice, the realized speedup is often limited by hard-to-draft positions. Even when most tokens within a speculative window are well modeled by the drafter, a single early mismatch can truncate the accepted prefix and invalidate the rest of the window, limiting overall throughput.

This behavior exposes a mismatch between the inference-time goal of speculative decoding and the training objectives commonly used for drafters. Recent methods, including MEDUSA [3], Hydra [4], EAGLE [5, 6, 7], HASS [8], and GRIFFIN [9], improve drafter quality through architectural design and supervised training, while distillation-based approaches [10, 11, 12] further improve distributional alignment with the target model. However, these methods are still primarily based on token-level supervised objectives. Such objectives improve local next-token prediction, but speculative utility is inherently *window-level* and *prefix-sensitive*: once the accepted prefix is truncated due to an early mismatch, the remaining drafted tokens in the speculative window are invalidated. As a result, improvements in token-level imitation may not consistently lead to longer accepted prefixes or higher speculative efficiency, especially when end-to-end performance is governed by a few acceptance bottlenecks.

Motivated by this mismatch, we propose **PPOW** (Performance-Driven Policy Optimization with Adaptive Windowing), a reinforcement learning framework that shifts drafter training from token-level supervision to window-level optimization over speculative windows (Figure 2). This window-level formulation aligns optimization with the prefix-sensitive structure induced by speculative

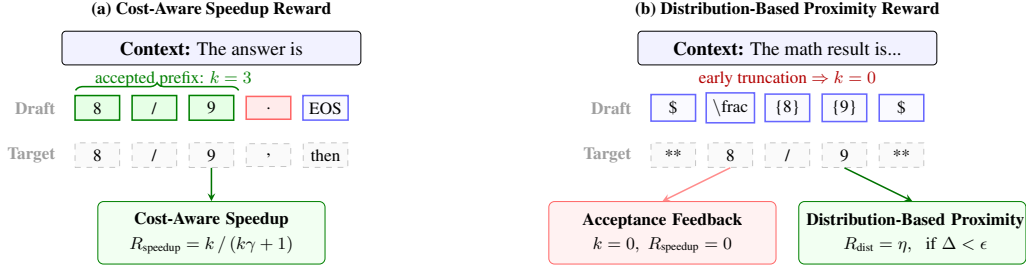


Figure 1: **PPOW uses a Cost-Aware Speedup Reward together with a Distribution-Based Proximity Reward.** (a) The Cost-Aware Speedup Reward increases with accepted prefix length and directly encourages speculative decoding efficiency. (b) When verification is truncated early, resulting in $k = 0$, the Distribution-Based Proximity Reward still provides auxiliary credit if the speculative window remains close to the target-preferred window under cumulative target log-likelihood.

verification. Concretely, PPOW combines a *Cost-Aware Speedup Reward*, which encourages longer accepted prefixes while accounting for relative drafter cost, with a *Distribution-Based Proximity Reward*, an auxiliary signal that preserves partial credit for speculative windows that remain close to the target model’s preferences even when the accepted prefix is truncated early.

Although PPOW optimizes with window-level rewards, not all speculative windows are equally informative during training. Treating all windows uniformly can disperse optimization effort across non-bottleneck windows, limiting focus on the acceptance bottlenecks most critical to speculative efficiency. PPOW therefore introduces *Adaptive Divergence-Aware Windowing* (ADAW), which prioritizes windows with large confidence-weighted draft–target divergence. This criterion is grounded in our analysis of potential acceptance bottlenecks in Appendix B, and our experiments show that prioritizing such windows improves speculative performance.

Our contributions are summarized as follows:

- **A Window-Level RL Framework for Drafter Optimization.** We formulate drafter training for speculative decoding as a reinforcement learning problem over speculative windows, which better matches inference-time acceptance behavior.
- **Performance-Driven Reward Design.** PPOW combines a Cost-Aware Speedup Reward with an auxiliary Distribution-Based Proximity Reward to better align training with speculative decoding. The latter provides additional credit for speculative windows that remain close to the target distribution, even when the accepted prefix is truncated early, as illustrated in Figure 1.
- **Adaptive Divergence-Aware Windowing.** We introduce a window prioritization strategy based on confidence-weighted draft–target divergence that focuses training on more informative windows associated with acceptance bottlenecks. Our analysis motivates this criterion, and our experiments show that prioritizing such windows improves speculative performance.

2 Related Work

Recent work has improved speculative decoding through advances in drafter design, draft–target alignment, and inference-time decoding optimization. One line of work improves the drafter itself. Head-based methods, such as MEDUSA [3] and Hydra [4], augment the model with auxiliary draft heads for multi-token prediction. Feature-based methods, including EAGLE [5, 6, 7], HASS [8], and GRIFFIN [9], improve drafting by leveraging hidden-state representations from the target model. Another line of work improves draft–target alignment through distillation or online adaptation, as in DistillSpec [10], OSD [11], and FastDraft [12]. Lookahead [13] provides a training-free alternative by constructing future candidates through Jacobi-style parallel decoding. While these methods substantially improve proposal quality, their objectives are typically defined at the token or local distribution level, leaving the window-level and prefix-sensitive nature of speculative verification less explicitly optimized.

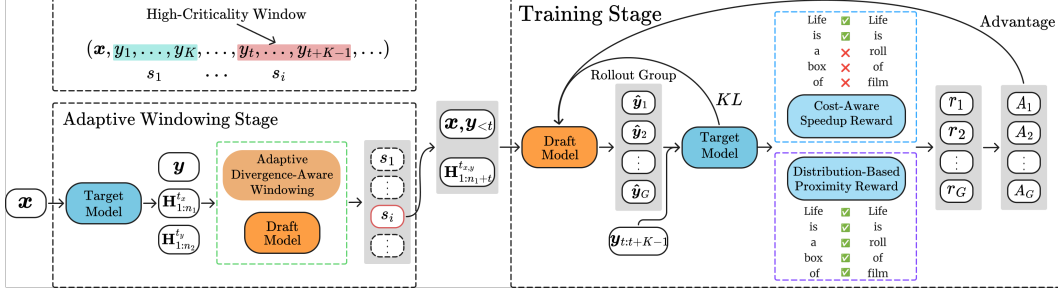


Figure 2: **Overview of PPOW.** PPOW performs policy optimization at the window level for speculative decoding. **Left:** Adaptive windowing uses confidence-weighted draft–target divergence scores to prioritize informative training windows. **Right:** The drafter samples a rollout group of speculative windows for policy optimization with performance-driven rewards and KL regularization.

Beyond drafter modeling, prior work also optimizes the speculative decoding pipeline at inference time, including multi-candidate selection and verification algorithms [14, 15], tree-based speculative inference and verification [16], tree-structured and hardware-aware speculation [17], cascaded or adaptive speculation strategies [18, 19, 20], and parallel draft–verify execution mechanisms [21, 22]. These approaches mainly operate on the inference procedure, such as how draft candidates are constructed, organized, or verified, and are complementary to drafter-training methods.

Recent studies have also explored reward signals or reinforcement learning in speculative decoding. RSD [23] uses an inference-time process reward model to guide draft acceptance, providing richer semantic guidance than standard token-level verification, without aiming to preserve the target-model distribution as in standard speculative verification. Another related direction includes Spec-RL [24], RLHFSpec [25], and ReSpec [26], which adapt speculative decoding to improve the efficiency of reinforcement learning rollout generation, whereas PPOW optimizes the drafter for standard speculative decoding inference.

Across these directions, PPOW differs by formulating drafter training as a reinforcement learning problem with performance-driven window-level objectives and adaptive prioritization of informative windows, targeting the window-level and prefix-sensitive utility of speculative verification while preserving the target-model distribution under standard speculative decoding protocol.

3 Preliminaries

We define the notation for speculative decoding used in this paper. Let π_{target} and π_{θ} denote the target model and the draft model (drafter), respectively, where θ parameterizes the drafter. Given a current prefix $\mathbf{x} = (x_1, \dots, x_{n_1})$, the drafter proposes a speculative window of K candidate tokens,

$$\hat{\mathbf{y}} = (\hat{y}_1, \dots, \hat{y}_K).$$

The target model then performs parallel verification with rejection sampling under the standard speculative decoding procedure, resulting in an accepted prefix of length $k \in \{0, \dots, K\}$.

3.1 Feature-Based Drafting

In this paper, we use feature-based drafters from the EAGLE family [5, 6, 7] as the base drafters in our experiments. Feature-based drafters augment token-based drafting with hidden-state features from the target model. Specifically, at drafting step t , the drafter predicts the next-token distribution as

$$\pi_{\theta}(\hat{y}_t \mid \mathbf{x}, \hat{\mathbf{y}}_{<t}, \mathbf{H}),$$

where

$$\mathbf{H} = [\mathbf{H}_{1:n_1}^{\text{tgt}}, \mathbf{H}_{<t}^d]$$

denotes the concatenation of the target model’s prefix hidden states $\mathbf{H}_{1:n_1}^{\text{tgt}}$ and the drafter’s hidden states $\mathbf{H}_{<t}^d$ within the current speculative window. Further details of the speculative verification procedure are provided in Appendix C.1.

3.2 Policy Optimization over Speculative Windows

In contrast to standard policy optimization formulations that operate on full responses, we formulate drafter training over fixed-length speculative windows. For each input prefix $\mathbf{x}$, the drafter defines a policy over speculative windows $\hat{\mathbf{y}} = (\hat{y}_1, \dots, \hat{y}_K)$, and a scalar reward $R(\mathbf{x}, \hat{\mathbf{y}})$ evaluates the window as a whole. The objective is

$$J(\theta) = \mathbb{E}_{\hat{\mathbf{y}} \sim \pi_\theta} [R(\mathbf{x}, \hat{\mathbf{y}})].$$

This formulation matches the speculative decoding procedure, in which a speculative window is verified as a whole and its utility is determined by the resulting accepted prefix. Appendix C provides further discussion of window-level policy optimization versus token-level training for speculative decoding.

4 PPOW

PPOW is a performance-driven training framework for speculative drafters that optimizes window-level behavior for speculative decoding. Concretely, PPOW uses Adaptive Divergence-Aware Windowing to prioritize informative speculative windows, assigns them rewards using the Cost-Aware Speedup Reward based on the accepted prefix length, complemented by the Distribution-Based Proximity Reward when accepted-prefix-based signals are sparse, and updates the drafter through window-level reinforcement learning. The framework is compatible with different drafter architectures. In our experiments, we instantiate PPOW with feature-based drafters from EAGLE-3 [7]. Figure 2 gives an overview of PPOW, and Algorithm 1 summarizes the full training procedure.

4.1 Window-Level Reinforcement Learning for Speculative Decoding

PPOW formulates drafter training as reinforcement learning over speculative windows, so as to better align optimization with the inference mechanism of speculative decoding. In speculative decoding, the drafter produces speculative windows and the target model verifies them in parallel, yielding accepted-prefix outcomes for different speculative windows under the same prefix. Since these outcomes determine speculative efficiency, PPOW optimizes at the window level and learns from the relative speculative utility of speculative windows.

For a given input prefix $\mathbf{x}$, PPOW samples G_{roll} speculative windows $\hat{\mathbf{y}} = (\hat{y}_1, \dots, \hat{y}_K)$ from the drafter policy, forming a rollout group for the same prefix. PPOW assigns each speculative window a scalar reward and normalizes rewards within the rollout group to produce group-relative advantages, thereby learning from the relative speculative utility of multiple windows sampled under the same context. Each speculative window is thus treated as a single training unit: it receives one window-level scalar reward, which is converted into a group-relative advantage. This grouped formulation is motivated by multi-candidate speculative decoding settings used in practice.

In addition, PPOW incorporates a KL regularization term into the objective. We treat the drafter as the policy π_θ and the target model π_{target} as a cross-scale distributional anchor, and use the KL divergence $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{target}})$ to encourage alignment during RL exploration. This target-anchored regularization is intended to stabilize training while keeping policy updates aligned with the target distribution used for speculative verification.

The resulting objective can be written as

$$J(\theta) = \frac{1}{G_{\text{roll}}} \sum_{i=1}^{G_{\text{roll}}} \frac{1}{K} \sum_{t=1}^K \left[\min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_i \right) - \beta D_{\text{KL}}(\pi_\theta(\cdot \mid \mathbf{x}, \hat{\mathbf{y}}_{i,<t}, \mathbf{H}) \parallel \pi_{\text{target}}(\cdot \mid \mathbf{x}, \hat{\mathbf{y}}_{i,<t})) \right],$$

where

$$r_{i,t}(\theta) = \frac{\pi_\theta(\hat{y}_{i,t} \mid \mathbf{x}, \hat{\mathbf{y}}_{i,<t}, \mathbf{H})}{\pi_{\text{old}}(\hat{y}_{i,t} \mid \mathbf{x}, \hat{\mathbf{y}}_{i,<t}, \mathbf{H})},$$

and $\hat{A}_i$ denotes the normalized group-relative advantage of the i -th speculative window, capturing its relative speculative utility within the rollout group. Specifically, for a rollout group of speculative

windows sampled under the same prefix, we compute

$$\hat{A}_i = \frac{R_i - \mu_R}{\sigma_R + \delta},$$

where R_i is the scalar reward of the i -th speculative window, μ_R and σ_R are the mean and standard deviation of rewards within the rollout group, and δ is a small positive constant for numerical stability.

Although the policy is autoregressively factorized over tokens, optimization is performed at the speculative-window level, with all tokens within the same window sharing a common advantage signal.

4.2 Adaptive Divergence-Aware Windowing

Window-level training better matches speculative decoding at inference time, but it also leads to substantial sample redundancy. A full response can be decomposed into many overlapping speculative windows, and many of them are already adequately modeled by a supervised-initialized drafter. At the same time, speculative performance is often constrained by certain hard-to-draft positions, where draft–target mismatch can more strongly affect the accepted prefix. PPOW therefore prioritizes speculative windows that are both more informative for training and more consequential for speculative acceptance length through Adaptive Divergence-Aware Windowing (ADAW).

Let P_t and Q_t denote the target and drafter distributions at position t ,

$$P_t = \pi_{\text{target}}(\cdot \mid \mathbf{x}, \mathbf{y}_{<t}), \quad Q_t = \pi_{\theta}(\cdot \mid \mathbf{x}, \mathbf{y}_{<t}, \mathbf{H}),$$

and define the token-level *criticality score* as

$$v_t = C(P_t) \cdot D_{\text{KL}}(P_t \parallel Q_t),$$

where

$$C(P_t) = 1 - \frac{H(P_t)}{\log |\mathcal{V}|},$$

and $\mathcal{V}$ denotes the vocabulary.

The criticality score v_t is a confidence-weighted measure of draft–target divergence. Here, $D_{\text{KL}}(P_t \parallel Q_t)$ captures mismatch between the drafter and target distributions, while $C(P_t)$ emphasizes contexts in which the target distribution is more concentrated and speculative verification is more sensitive to such mismatch. In this way, ADAW emphasizes contexts that are both difficult for the drafter and more consequential for speculative acceptance.

Because speculative decoding operates on complete windows, PPOW aggregates token-level criticality over each speculative window:

$$s_j = \frac{1}{K} \sum_{t=j}^{j+K-1} v_t,$$

and prioritizes windows with larger s_j during training. This adaptive windowing strategy reduces redundancy while focusing optimization on windows more directly associated with acceptance bottlenecks. Appendix B analyzes the connection between draft–target divergence and speculative acceptance behavior, and further discusses the role of confidence weighting in prioritizing training windows.

4.3 Performance-Driven Rewards

PPOW uses a window-level reward with two components. The primary component is a Cost-Aware Speedup Reward that encourages longer accepted prefixes while accounting for both drafting and verification cost. The second component is a complementary Distribution-Based Proximity Reward that provides auxiliary credit when exact verification terminates early but the speculative window still achieves a similar cumulative target-model log-likelihood.

4.3.1 Cost-Aware Speedup Reward

The primary goal of speculative decoding is to improve inference efficiency. PPOW therefore uses a cost-aware reward based on the accepted prefix length k of a speculative window and the relative cost γ of the drafter:

$$R_{\text{speedup}} = \frac{k}{k\gamma + 1},$$

where $k \in \{0, \dots, K\}$ is the accepted length and γ denotes the relative computational cost of π_θ with respect to π_{target} , estimated in our implementation by the ratio of non-embedding parameters.

This formulation accounts for the computational structure of speculative decoding by balancing accepted length against relative drafting cost. The term $k\gamma + 1$ combines the drafting cost and the target-model verification cost within a speculative step. Compared with using the raw accepted prefix length k alone, R_{speedup} provides a cost-aware objective intended to better reflect the efficiency trade-off in speculative decoding.

We use this formulation during training instead of rewards derived from directly measured speedup, since measured speedup depends on the execution environment and is therefore less suitable as a general speculative-window-level training reward. Appendix D compares the Cost-Aware Speedup Reward with a Measured-Speedup-Based alternative and shows that it preserves the same acceptance-related trend while remaining effective for optimization.

4.3.2 Distribution-Based Proximity Reward

The accepted-prefix reward alone can become sparse when verification terminates early, even if the drafted window remains broadly compatible with the target model’s preferences. To provide auxiliary partial credit in such cases, PPOW introduces the Distribution-Based Proximity Reward, which compares the drafted window with a target-preferred window under the target model’s cumulative log-likelihood. Figure 1(b) illustrates an example of such early truncation, where verification yields $k = 0$ because the drafter and target model diverge at the token level.

For a drafted speculative window $\hat{\mathbf{y}}$, we construct a target-preferred reference window $\mathbf{y}$ autoregressively from the target model under the same context, and compare the cumulative target log-probabilities of the two windows:

$$\Delta = \sum_{t=1}^K \left[\log \pi_{\text{target}}(y_t \mid \mathbf{x}, \mathbf{y}_{<t}) - \log \pi_{\text{target}}(\hat{y}_t \mid \mathbf{x}, \hat{\mathbf{y}}_{<t}) \right],$$

where

$$y_t = \arg \max_y \pi_{\text{target}}(y \mid \mathbf{x}, \mathbf{y}_{<t}).$$

We then define

$$R_{\text{dist}} = \eta \cdot \mathbf{1}[\Delta < \epsilon],$$

where ϵ is a tolerance threshold and η scales the contribution of this reward.

This auxiliary reward is designed to capture training signal that may be missed by accepted-prefix-based reward alone. In PPOW, R_{dist} is activated only when verification yields no accepted token, providing bounded partial credit to drafted windows whose cumulative target log-probabilities remain close to those of the target-preferred reference window.

As a result, R_{dist} complements the accepted-prefix reward with a softer window-level signal and can provide denser feedback during training. Section 5.5 evaluates its overall contribution in ablations, and Appendix E further examines its effect on easy and hard windows.

5 Experiments

We evaluate PPOW from four perspectives: (1) speculative decoding performance across model families and tasks under a unified decoding protocol, (2) practical efficiency under different inference candidate group sizes, (3) whether PPOW outperforms continued supervised training with the same number of post-training steps, (4) ablations and stability studies of the reward design and Adaptive Divergence-Aware Windowing. Additional results on the speedup proxy, easy/hard-window behavior, and broader baseline comparisons are provided in the appendix.

5.1 Experimental Setup

Models and Tasks. We evaluate PPOW on two representative model families: LLaMA-3 [27] with 8B and 70B target models, and Qwen3 [28] with 8B and 32B target models. PPOW is applied to feature-based drafters from the EAGLE family [5, 6, 7] and initialized from supervised checkpoints before RL optimization. We evaluate on MT-Bench [29] for multi-turn dialogue, HumanEval [30] for code generation, and GSM8K [31] for mathematical reasoning.

Metrics. We evaluate speculative decoding performance using the following two metrics:

- **Speedup Ratio:** The measured end-to-end speedup relative to vanilla autoregressive decoding.
- **Average Acceptance Length (τ):** The average number of tokens accepted from the drafter in each speculative verification step.

Unless otherwise specified, all reported results are obtained with decoding temperature set to 0.0.

Baselines. Our main comparisons are with learned drafters, including EAGLE-3 [7] and GRIF-FIN [9]. Additional results on broader natural-language tasks and supplementary comparisons with OSD [11], Lookahead [13], and FastDraft [12] are provided in Appendix F. Detailed optimization hyperparameters and the unified decoding protocol are provided in Appendix A.

5.2 Main Results

Table 1 summarizes the main results. PPOW achieves the best mean performance over the evaluated benchmarks for each model family under both temperature settings.

Table 1: **Main speculative decoding results.** Average acceptance length (τ) and speedup across models and benchmarks under a unified decoding protocol. L31, L33, and Q3 refer to LLaMA-3.1-Instruct, LLaMA-3.3-Instruct, and Qwen3, respectively.

Model	Method	MT-Bench		HumanEval		GSM8K		Mean	
		τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup
Temperature = 0.0									
L31-8B	GRIFFIN	5.14	2.58×	6.72	3.96×	5.98	3.38×	5.95	3.31×
	EAGLE-3	5.53	2.91 ×	6.63	3.92×	6.12	3.41×	6.09	3.41×
	PPOW	5.47	2.72×	7.23	4.14 ×	6.50	3.52 ×	6.40	3.46 ×
L33-70B	GRIFFIN	5.08	3.60×	6.69	4.78×	5.90	3.99×	5.89	4.12×
	EAGLE-3	5.12	3.63×	6.78	4.80×	5.93	4.02×	5.94	4.15×
	PPOW	5.45	3.73 ×	6.96	4.82 ×	6.47	4.54 ×	6.29	4.36 ×
Q3-8B	EAGLE-3	4.95	2.64×	6.68	3.40×	6.86	3.47×	6.16	3.17×
	PPOW	5.58	3.02 ×	7.01	3.62 ×	6.97	3.54 ×	6.52	3.39 ×
Q3-32B	EAGLE-3	5.25	3.47×	6.52	4.02×	6.21	3.94×	5.99	3.81×
	PPOW	5.78	3.54 ×	6.91	4.16 ×	6.62	4.05 ×	6.44	3.92 ×
Temperature = 1.0									
L31-8B	GRIFFIN	4.03	2.24×	6.01	3.29×	5.13	2.89×	5.06	2.81×
	EAGLE-3	4.17	2.35 ×	5.94	3.25×	5.16	2.89×	5.09	2.83×
	PPOW	4.12	2.29×	6.33	3.37 ×	5.63	3.13 ×	5.36	2.93 ×
L33-70B	GRIFFIN	4.70	3.38×	6.31	4.46×	5.54	3.87×	5.52	3.90×
	EAGLE-3	4.82	3.40×	6.28	4.43×	5.98	4.16×	5.69	4.00×
	PPOW	5.14	3.52 ×	6.57	4.62 ×	6.29	4.49 ×	6.00	4.21 ×
Q3-8B	EAGLE-3	4.00	2.41×	5.71	2.11×	5.52	2.89×	5.08	2.47×
	PPOW	4.72	2.82 ×	5.96	2.33 ×	5.76	3.01 ×	5.48	2.72 ×
Q3-32B	EAGLE-3	4.51	2.60×	5.53	2.59×	5.23	2.81×	5.09	2.67×
	PPOW	4.95	2.81 ×	6.17	2.77 ×	5.61	3.01 ×	5.58	2.86 ×

Across model families, PPOW consistently improves average acceptance length and wall-clock speedup over strong learned-drafter baselines under the unified decoding protocol. The gains are particularly clear on HumanEval and GSM8K, where PPOW delivers the strongest and most consistent improvements across both LLaMA and Qwen models, suggesting that it is especially effective when speculative success depends on relatively structured decoding decisions. By contrast, the trend is less pronounced on MT-Bench, likely because open-ended dialogue admits a broader range of valid continuations, making speculative acceptance less sensitive to improvements in the drafter policy.

5.3 Inference Candidate Group-Size Trade-offs

Table 2: **Average acceptance length (τ) under different inference candidate group sizes.** PPOW shows consistently higher acceptance length than the supervised baseline with smaller candidate group sizes on LLaMA-3.1-8B / GSM8K.

Candidate Size	PPOW	Supervised baseline
1	4.92	4.10
2	5.23	4.46
4	6.33	5.03
8	6.41	5.58
16	6.49	6.12

Table 3: **Average acceptance length (τ) after continued training.** PPOW achieves higher final acceptance length than EAGLE-3 with continued supervised training (CST) across learning rates on GSM8K with LLaMA-3.1-8B.

Method	LR	Steps	τ
EAGLE-3 (CST)	1e-5	30k	5.72
	5e-5	30k	5.47
PPOW	1e-5	30k	6.50
	5e-5	30k	6.29

Table 2 compares PPOW with the corresponding supervised baseline under different inference candidate group sizes. Here, the supervised baseline refers to the EAGLE-3 [7] initialization without PPOW RL post-training. In practical speculative decoding, acceptance can often be improved by generating multiple candidate token sequences—for example through branching or tree-style drafting—and then verifying them in parallel. Larger candidate groups therefore tend to improve acceptance, but they also increase verification overhead. PPOW reaches a high acceptance length with much smaller candidate groups: at inference candidate group size 4, it achieves $\tau = 6.33$, whereas the supervised baseline reaches only $\tau = 5.03$ under the same setting and requires a much larger candidate group size of 16 to approach PPOW’s performance. This result suggests that PPOW uses the candidate budget more effectively under the same verification budget.

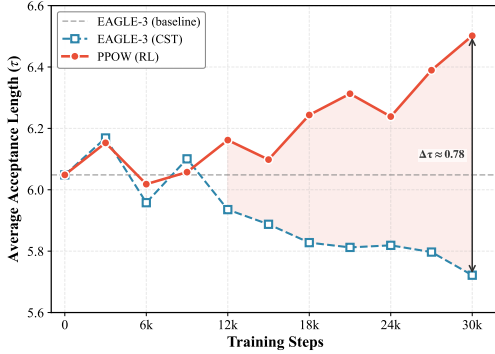


Figure 3: **PPOW versus continued supervised training under matched training steps.** On GSM8K with LLaMA-3.1-8B, the supervised baseline initially improves average acceptance length but later degrades, whereas PPOW continues to improve and achieves a higher final acceptance length. CST denotes continued supervised training from the EAGLE-3 checkpoint.

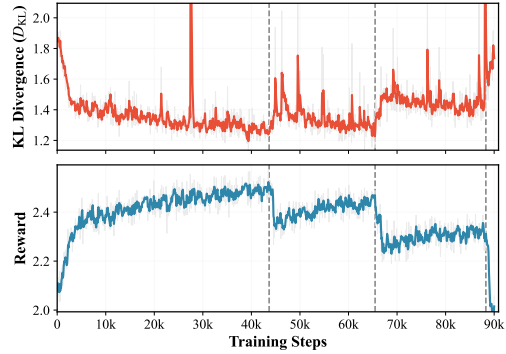


Figure 4: **Training dynamics after enabling ADAW.** Switching to ADAW at 44k training steps causes an immediate drop in training reward and a corresponding rise in KL divergence, indicating that the sampled windows have become more challenging. As training continues, the policy adapts to these harder windows and gradually recovers performance.

5.4 PPOW versus Continued Supervised Training

We next compare PPOW against a continued supervised training baseline to determine whether its gains can be explained by additional supervised training alone. Both methods are initialized from the same EAGLE-3 checkpoint [7] and trained for an identical number of additional steps. Figure 3 shows that, on GSM8K with LLaMA-3.1-8B [27], the supervised baseline initially improves average acceptance length but later degrades, whereas PPOW continues to improve throughout training and reaches a higher final value. Table 3 shows that the same trend holds across learning rates after 30k additional training steps. These results suggest that simply extending imitation-style training does not reliably optimize speculative acceptance behavior. The objective-level difference between PPOW and the supervised baseline is further discussed in Appendix C. We observe the same qualitative trend on natural-language tasks, as shown in Appendix F.2.

5.5 Ablation Studies

We examine the contribution of PPOW’s two key components beyond the base speedup reward: the Distribution-Based Proximity Reward and Adaptive Divergence-Aware Windowing. Table 4 reports component ablations on LLaMA-3.1-8B.

Table 4: **Ablation of PPOW components on LLaMA-3.1-8B.** “w/o R_{dist} ” removes the Distribution-Based Proximity Reward, “w/o ADAW” replaces Adaptive Divergence-Aware Windowing with uniform window sampling, and “w/o both” removes R_{dist} and uses uniform window sampling.

Method	MT-Bench		GSM8K	
	τ	Speedup	τ	Speedup
w/o R_{dist}	5.05	2.52×	6.41	3.48×
w/o ADAW	4.82	2.44×	6.35	3.37×
w/o both	4.38	2.39×	6.05	3.26×
PPOW (Full)	5.47	2.72×	6.50	3.52×

Removing either R_{dist} or ADAW reduces both acceptance length and end-to-end speedup, and removing both leads to the largest drop on both benchmarks. On MT-Bench, removing both reduces τ from 5.47 to 4.38 and speedup from 2.72× to 2.39×. This pattern suggests that the two components make complementary contributions in practice: R_{dist} provides a denser learning signal when exact verification is sparse, while ADAW improves training efficiency by focusing optimization on more informative windows.

Figure 4 provides further evidence for the effectiveness of ADAW. When training switches to ADAW at 44k steps, reward drops immediately while KL divergence rises, indicating that training shifts toward harder windows with larger draft–target divergence. As optimization continues, the policy adapts to this more difficult window distribution and recovers performance. This behavior is consistent with ADAW prioritizing harder-to-draft windows rather than repeatedly sampling easy or redundant ones. Additional analysis of ADAW is provided in Appendix B, which gives analytical support for weighting draft–target divergence by target confidence. Appendix E provides additional ablations and an easy/hard-window analysis of the Distribution-Based Proximity Reward.

6 Conclusion

We presented PPOW, a performance-driven reinforcement learning framework for speculative decoding that optimizes the drafter over speculative windows. PPOW combines the Cost-Aware Speedup Reward, the Distribution-Based Proximity Reward, and Adaptive Divergence-Aware Windowing to better align drafter optimization with inference-time acceptance behavior. Empirically, PPOW improves average acceptance length and speedup across diverse settings. It also remains effective with substantially smaller inference candidate group sizes, suggesting practical value under constrained verification budgets. We further provide analytical support for the confidence-weighted draft–target divergence criterion used in ADAW. Overall, our results provide strong evidence for the value of performance-driven drafter optimization in improving speculative decoding efficiency.

References

- [1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.
- [2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.
- [4] Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024.
- [5] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.
- [6] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 7421–7432, 2024.
- [7] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.
- [8] Lefan Zhang, Xiaodan Wang, Yanhua Huang, and Ruiwen Xu. Learning harmonized representations for speculative sampling. *arXiv preprint arXiv:2408.15766*, 2024.
- [9] Shijing Hu, Jingyang Li, Xingyu Xie, Zhihui Lu, Kim-Chuan Toh, and Pan Zhou. Griffin: Effective token alignment for faster speculative decoding. *arXiv preprint arXiv:2502.11018*, 2025.
- [10] Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. Distillspec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461*, 2023.
- [11] Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Online speculative decoding. *arXiv preprint arXiv:2310.07177*, 2023.
- [12] Ofir Zafir, Igor Margulis, Dorin Shteyman, Shira Guskin, and Guy Boudoukh. Fastdraft: How to train your draft. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22488–22505, 2025.
- [13] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.
- [14] Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. Spectr: Fast speculative decoding via optimal transport. *Advances in Neural Information Processing Systems*, 36:30222–30242, 2023.
- [15] Ziteng Sun, Uri Mendlovic, Yaniv Leviathan, Asaf Aharoni, Jae Hun Ro, Ahmad Beirami, and Ananda Theertha Suresh. Block verification accelerates speculative decoding. *arXiv preprint arXiv:2403.10444*, 2024.
- [16] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949, 2024.

- [17] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, robust, and hardware-aware speculative decoding. *arXiv preprint arXiv:2402.12374*, 2024.
- [18] Ziyi Chen, Xiaocong Yang, Jiacheng Lin, Chenkai Sun, Kevin C Chang, and Jie Huang. Cascade speculative drafting for even faster llm inference. *Advances in Neural Information Processing Systems*, 37:86226–86242, 2024.
- [19] Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe Berchansky, Nadav Timor, Moshe Wasserblat, and Roy Schwartz. Dynamic speculation lookahead accelerates speculative decoding of large language models. *arXiv preprint arXiv:2405.04304*, 2024.
- [20] Xukun Liu, Bowen Lei, Ruqi Zhang, and Dongkuan DK Xu. Adaptive draft-verification for efficient large language model decoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24668–24676, 2025.
- [21] Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin. Specexec: Massively parallel speculative decoding for interactive llm inference on consumer devices. *Advances in Neural Information Processing Systems*, 37:16342–16368, 2024.
- [22] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. Pearl: Parallel speculative decoding with adaptive draft length. *arXiv preprint arXiv:2408.11850*, 2024.
- [23] Baohao Liao, Yuhui Xu, Hanze Dong, Junnan Li, Christof Monz, Silvio Savarese, Doyen Sahoo, and Caiming Xiong. Reward-guided speculative decoding for efficient llm reasoning. *arXiv preprint arXiv:2501.19324*, 2025.
- [24] Bingshuai Liu, Ante Wang, Zijun Min, Liang Yao, Haibo Zhang, Yang Liu, Anxiang Zeng, and Jinsong Su. Spec-rl: Accelerating on-policy reinforcement learning via speculative rollouts. *arXiv preprint arXiv:2509.23232*, 2025.
- [25] Siqi Wang, Hailong Yang, Junjie Zhu, Xuezhu Wang, Yufan Xu, and Depei Qian. Rlhf-spec: Breaking the efficiency bottleneck in rlhf training via adaptive drafting. *arXiv preprint arXiv:2512.04752*, 2025.
- [26] Qiaoling Chen, Zijun Liu, Peng Sun, Shenggui Li, Guoteng Wang, Ziming Liu, Yonggang Wen, Siyuan Feng, and Tianwei Zhang. Respec: Towards optimizing speculative decoding in reinforcement learning systems. *arXiv preprint arXiv:2510.26475*, 2025.
- [27] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [28] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [29] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.
- [30] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [31] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [32] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3029–3051, 2023.

- [33] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.
- [34] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [35] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [36] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [37] Shashi Narayan, Shay B Cohen, and Mirella Lapata. Don’t give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 1797–1807, 2018.
- [38] Ondřej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, et al. Findings of the 2014 workshop on statistical machine translation. In *Proceedings of the ninth workshop on statistical machine translation*, pages 12–58, 2014.

A Optimization and Implementation Details

This appendix summarizes the training setup, optimization procedure, hyperparameter settings, and full PPOW training algorithm used in our experiments.

A.1 Training Setup

Our training uses a two-stage pipeline: supervised initialization followed by PPOW-based reinforcement learning. For LLaMA-3, we initialize from the official EAGLE-3 checkpoints [7]. For Qwen, we first train an EAGLE-3 drafter on ShareGPT and UltraChat-200k [32], and then further optimize it with PPOW on the same data mixture. All experiments are run on NVIDIA H100 (80GB) GPUs using PyTorch with FSDP [33] to support a frozen target model and a trainable drafter. PPOW training costs about 50, 100, and 200 GPU-hours for 8B, 32B, and 70B targets, respectively.

A.2 Policy Optimization and Target-Anchored KL Regularization

PPOW uses a group-relative clipped policy objective [34, 35]. For each selected speculative window, we sample a rollout group of G_{roll} drafts, compute a scalar reward for each draft, and normalize rewards within the group to obtain group-relative advantages.

A key difference from standard RL training setups lies in the KL regularization. Rather than constraining the current policy toward an initialization policy or an auxiliary reference model, PPOW computes KL regularization between the drafter policy and the frozen target model. Specifically, we compute the token-wise KL divergence between the draft and target distributions at each drafted position and average it over the speculative window. This keeps exploration anchored to the distribution that ultimately governs speculative verification.

A.3 Training and Decoding Configurations

Table 5 summarizes the default training hyperparameters, hard-window curriculum schedule, and decoding settings used throughout our experiments. Under hard-window curriculum scheduling, PPOW begins with a smaller proportion of ADAW-selected hard windows and gradually increases this proportion over the course of training.

Table 5: Training hyperparameters, curriculum schedule, and decoding configurations used in our experiments.

Hyperparameters	Value
<i>Training Configuration</i>	
Clip ratio ϵ_{clip}	0.2
KL coefficient β	0.03
Rollout group size G_{roll}	8
Speculative window size K	10
Relative computational cost γ	0.12
R_{dist} tolerance threshold ϵ	0.5
R_{dist} scaling factor η	1.0
Hard-window curriculum	0.2 / 0.4 / 0.6
Learning rate	5×10^{-6}
Warmup ratio	0.05
Precision	bf16
<i>Decoding Configurations</i>	
Decoding strategy	tree decoding
KV cache	true
Speculative depth / num_steps	10
Branching factor / top-k	3
Verification rule	rejection sampling
Temperatures	0.0 / 1.0

A.4 Full PPOW Training Algorithm

Algorithm 1 PPOW Training Procedure

```

1: Inputs: drafter  $\pi_\theta$ , target model  $\pi_{\text{target}}$ , speculative window size  $K$ , rollout group size  $G_{\text{roll}}$ , clip
   ratio  $\epsilon_{\text{clip}}$ , KL coefficient  $\beta$ , relative cost  $\gamma$ , tolerance threshold  $\epsilon$ , scaling factor  $\eta$ 
2: for each training prefix  $\mathbf{x}$  do
3:    $\triangleright$  Generate target-side outputs and draft distributions
4:    $(\mathbf{y}, \mathbf{H}, \{P_t\}_{t=1}^T) \leftarrow \pi_{\text{target}}(\mathbf{x})$ 
5:    $\{Q_t\}_{t=1}^T \leftarrow \pi_\theta(\mathbf{x}, \mathbf{y}, \mathbf{H})$ 
6:    $\triangleright$  Compute criticality score
7:   for  $t = 1, \dots, T$  do
8:      $v_t \leftarrow \left(1 - \frac{H(P_t)}{\log |\mathcal{V}|}\right) \cdot D_{\text{KL}}(P_t \parallel Q_t)$ 
9:   end for
10:   $\triangleright$  ADAW-based window selection
11:  for each window start index  $j \in \{1, \dots, T - K + 1\}$  do
12:     $s_j \leftarrow \frac{1}{K} \sum_{t=j}^{j+K-1} v_t$ 
13:  end for
14:  Sample a training window start  $j^*$  from the normalized weights proportional to  $\{s_j\}$ 
15:   $\triangleright$  Grouped rollout and reward computation
16:  for  $g = 1, \dots, G_{\text{roll}}$  do
17:    Roll out a speculative window  $\hat{\mathbf{y}}_g \sim \pi_\theta(\cdot \mid \mathbf{x}, \mathbf{y}_{<j^*}, \mathbf{H}_{<j^*})$ 
18:     $k_g \leftarrow \text{VERIFY}(\hat{\mathbf{y}}_g, \pi_{\text{target}})$ 
19:    Compute the proximity gap  $\Delta_g$  for  $\hat{\mathbf{y}}_g$  under the same base context  $(\mathbf{x}, \mathbf{y}_{<j^*})$  as defined in
    Section 4.3.2
20:     $r_g \leftarrow \frac{k_g}{k_g \gamma + 1} + \eta \cdot \mathbf{1}[k_g = 0] \mathbf{1}[\Delta_g < \epsilon]$ 
21:  end for
22:  Compute normalized group-relative advantages  $\{\hat{A}_g\}_{g=1}^{G_{\text{roll}}}$  from rewards  $\{r_g\}_{g=1}^{G_{\text{roll}}}$ 
23:   $\triangleright$  Policy update
24:  Update  $\theta$  using the clipped group-relative policy objective with target-anchored KL regulariza-
  tion
25: end for

```

B Analysis of Confidence-Weighted Draft–Target Divergence

In Section 4.2, we define the token-level *criticality score*

$$v_t = C(P_t) D_{\text{KL}}(P_t \parallel Q_t),$$

where P_t and Q_t denote the target and drafter distributions at position t , and $C(P_t)$ is an entropy-normalized confidence score of the target distribution. In the main text, this score serves as a prioritization signal for training windows. This appendix provides analytical support for this design by relating draft–target divergence to speculative acceptance behavior and clarifying the role of confidence weighting.

B.1 Acceptance Probability and Draft–Target Divergence

Let α_t denote the expected acceptance probability at position t under speculative decoding. Following [1], when a draft token is sampled from Q_t and verified against P_t , the expected acceptance probability is

$$\begin{aligned}
\alpha_t &= \mathbb{E}_{y \sim Q_t} \left[\min \left(1, \frac{P_t(y)}{Q_t(y)} \right) \right] \\
&= \sum_{y \in \mathcal{V}} \min(P_t(y), Q_t(y)).
\end{aligned}$$

Using the identity $\min(a, b) = \frac{1}{2}(a + b - |a - b|)$, we obtain

$$\begin{aligned}\alpha_t &= \frac{1}{2} \sum_{y \in \mathcal{V}} (P_t(y) + Q_t(y) - |P_t(y) - Q_t(y)|) \\ &= 1 - \frac{1}{2} \sum_{y \in \mathcal{V}} |P_t(y) - Q_t(y)| \\ &= 1 - \delta(P_t, Q_t),\end{aligned}$$

where $\delta(P_t, Q_t)$ denotes the total variation distance.

Applying Pinsker’s inequality,

$$\delta(P_t, Q_t) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(P_t \parallel Q_t)},$$

yields

$$\alpha_t \geq 1 - \sqrt{\frac{1}{2} D_{\text{KL}}(P_t \parallel Q_t)}.$$

This bound shows that smaller draft–target KL divergence implies a higher lower bound on the acceptance probability. It therefore supports the use of draft–target divergence as a proxy of speculative difficulty.

B.2 Confidence Modulation of Draft–Target Divergence

The analysis above shows that draft–target divergence is informative about acceptance behavior. However, the practical significance of a given divergence also depends on the uncertainty of the target distribution. For the same level of draft–target divergence, discrepancies are typically more consequential when the target distribution is concentrated than when the target itself is uncertain.

To account for this effect, we introduce the confidence factor

$$C(P_t) = 1 - \frac{H(P_t)}{\log |\mathcal{V}|} \in [0, 1],$$

which assigns larger values to lower-entropy target distributions and smaller values to higher-entropy ones.

The resulting criticality score,

$$v_t = C(P_t) D_{\text{KL}}(P_t \parallel Q_t),$$

can be viewed as a confidence-modulated measure of draft–target divergence. It increases the priority of positions where the drafter distribution diverges from a confident target distribution, while reducing the influence of divergence in higher-entropy contexts. Aggregating v_t over a speculative window therefore yields a window-level signal that emphasizes regions more likely to constrain speculative acceptance in practice.

C Optimization Differences Between PPOW and Continued Supervised Training

PPOW and continued supervised training target different optimization objectives in speculative decoding. Continued supervised training improves token-level imitation of the target model, whereas PPOW directly optimizes the window-level utility aligned with speculative decoding efficiency at inference time.

C.1 Speculative Verification and Window-Level Utility

Speculative decoding commonly uses rejection-sampling verification to preserve the target-model distribution during inference. Under this verification rule, the practical gain of a speculative window depends on how many of its tokens are accepted by the target model, making the accepted prefix length, rather than token-level likelihood alone, the relevant measure of speculative utility.

For a speculative window $\hat{\mathbf{y}} = (\hat{y}_1, \dots, \hat{y}_K)$, the acceptance probability of the token at position t under rejection-sampling verification is

$$\alpha_t = \min \left(1, \frac{\pi_{\text{target}}(\hat{y}_t \mid \mathbf{x}, \hat{\mathbf{y}}_{<t})}{\pi_{\theta}(\hat{y}_t \mid \mathbf{x}, \hat{\mathbf{y}}_{<t})} \right).$$

The acceptance length τ is then determined by the first position at which verification fails:

$$\tau = \sum_{n=1}^K \prod_{t=1}^n \mathbf{1}[u_t \leq \alpha_t], \quad u_t \sim U(0, 1).$$

Because τ is defined by consecutive acceptance from the beginning of the speculative window, early truncation under verification immediately eliminates the contribution of all subsequent tokens in that window. In this sense, speculative utility is inherently window-level, and positions that cause early verification truncation can become bottlenecks with disproportionate impact on inference-time efficiency. This creates a mismatch with token-level training objectives and motivates the window-level optimization used in PPOW.

C.2 Limitations of Continued Supervised Training for Speculative Decoding

Continued supervised training retains the same token-level cross-entropy objective as standard supervised training, and this token-level training objective is not fully aligned with the window-level speculative utility relevant at inference time:

$$\begin{aligned} \mathcal{L}_{\text{sup}} &= \mathbb{E}_{\mathbf{y} \sim \pi_{\text{target}}} [-\log \pi_{\theta}(\mathbf{y} \mid \mathbf{x})] \\ &= \mathbb{E}_{\mathbf{y} \sim \pi_{\text{target}}} \left[\sum_{t=1}^K -\log \pi_{\theta}(y_t \mid \mathbf{x}, \mathbf{y}_{<t}) \right]. \end{aligned}$$

This objective improves draft–target imitation. However, its effect on speculative decoding is indirect, because the optimized quantity is token-level likelihood rather than the realized utility of a speculative window. In other words, matching the target model token by token does not necessarily maximize the number of tokens that survive verification within an accepted speculative window.

As a result, continued supervised training does not directly optimize accepted prefix length, focus explicitly on bottleneck positions that cause early verification truncation, or optimize across multiple sampled speculative windows for the same context. These mismatches suggest that continued supervised training may provide only limited additional gains for inference-time speculative efficiency, especially once token-level imitation has largely saturated, and further motivate the window-level optimization used in PPOW.

C.3 PPOW as Window-Level Policy Optimization

PPOW formulates speculative decoding as a policy optimization problem, where the drafter is optimized over speculative windows using window-level rewards aligned with inference-time speculative performance (Section 4.3):

$$J(\theta) = \mathbb{E}_{\hat{\mathbf{y}} \sim \pi_{\theta}} [R(\mathbf{x}, \hat{\mathbf{y}})].$$

Here, optimization is driven by the realized reward of sampled speculative windows rather than by token-level supervision on a fixed target token sequence. Since a full response can be decomposed into many overlapping speculative windows, not all windows contribute equally informative training signals. Window-level optimization therefore makes it possible to focus learning on more informative speculative windows, especially those containing positions that are more critical to early truncation under verification.

PPOW also samples multiple speculative windows for the same context and computes group-relative advantages among them. This grouped optimization reflects practical settings that evaluate multiple candidate token sequences during decoding and provides a stronger relative learning signal toward speculative windows with better acceptance behavior.

D Cost-Aware Speedup Reward versus Measured Speedup Reward

A natural alternative to the Cost-Aware Speedup Reward in Section 4.3 is to define training rewards directly from measured inference speedup. In practice, however, measured speedup depends strongly on the serving environment, including hardware, backend implementation, cache behavior, batching, and verification configuration. It can also provide noisy or low-resolution feedback at the speculative-window level. This appendix compares the Cost-Aware Speedup Reward and a Measured-Speedup-Based Reward alternative from two perspectives: whether they exhibit similar reward trends as accepted length increases, and whether training with measured speedup yields meaningful gains in a fixed setup.

Table 6: **Reward values induced by accepted prefix length.** Comparison between a Measured-Speedup-Based Reward and PPOW’s Cost-Aware Speedup Reward as accepted length increases. The two rewards are not numerically identical, but they preserve the same monotonic trend.

Accepted Prefix Length	Measured Speedup Reward	Cost-Aware Speedup Reward
1	0.81	0.89
2	1.49	1.60
3	1.81	2.18
4	2.06	2.67
5	2.49	3.08
6	2.62	3.43
7	2.88	3.74

Table 7: **Training with PPOW’s Cost-Aware Speedup Reward vs. Measured-Speedup-Based Reward.** Final average acceptance length (τ) on GSM8K with LLaMA-3.1-8B under the same training budget.

Reward formulation	Average acceptance length (τ)
Cost-Aware Speedup	6.50
Measured Speedup	6.61

Table 6 compares the reward values induced by accepted prefix length under the two formulations. Although the Cost-Aware Speedup Reward does not numerically match the Measured-Speedup-Based Reward, the two exhibit the same monotonic trend: longer accepted prefixes receive larger reward under both formulations. The two rewards differ in scale because the measured reward is system-dependent, but they preserve the same ordering over accepted-prefix outcomes. This shared trend is important for training, since it preserves the relative preference for speculative windows with better efficiency characteristics.

Table 7 compares training with the Cost-Aware Speedup Reward against training with a Measured-Speedup-Based Reward under the same budget. In this fixed setup, the measured-speedup-based reward yields only a small improvement in final acceptance length. However, such a reward remains tightly coupled to the specific serving stack used during training and may provide noisy or low-resolution feedback at the window level. The cost-aware formulation therefore offers a more portable and practical default objective.

Overall, these results support the use of the Cost-Aware Speedup Reward during training: it tracks the same acceptance-related trend as measured speedup and remains effective for optimization, while avoiding the system dependence of direct runtime-based rewards.

E Effect of the Distribution-Based Proximity Reward Across Easy and Hard Windows

To characterize the effect of the Distribution-Based Proximity Reward R_{dist} , we partition speculative windows according to the acceptance behavior of the supervised baseline. We refer to windows with full acceptance ($k = K$) as *easy*, and to those for which verification terminates before the end of the speculative window ($k < K$) as *hard*.

Table 8: **Effect of the Distribution-Based Proximity Reward on easy and hard windows.** Easy windows are fully accepted by the supervised baseline ($k = K$), whereas hard windows terminate early within the speculative window ($k < K$). Adding R_{dist} preserves performance on easy windows and further improves acceptance length while reducing the alignment divergence metric ∇ on hard windows.

Sample Type	Model	τ	∇
Easy	Supervised Baseline	6.0	0.009
	PPOW w/o R_{dist}	6.0	0.012
	PPOW (Full)	6.0	0.017
Hard	Supervised Baseline	2.32	7.091
	PPOW w/o R_{dist}	4.58	3.644
	PPOW (Full)	4.86	3.275

Under this partition, we compare the supervised baseline, PPOW without R_{dist} , and full PPOW. Table 8 reports both average acceptance length τ and an alignment divergence metric ∇ , defined token-wise as

$$\nabla = \exp(\delta) - \delta - 1, \quad \delta = \log \pi_{\text{target}}(\hat{y}_t \mid \cdot) - \log \pi_{\theta}(\hat{y}_t \mid \cdot).$$

This metric measures draft–target alignment on drafted tokens through token-level log-probability differences, with lower values indicating better alignment.

On easy windows, the supervised baseline, PPOW without R_{dist} , and full PPOW have identical acceptance length, and their ∇ values differ only slightly. On hard windows, however, full PPOW improves acceptance length ($4.58 \rightarrow 4.86$) while further reducing ∇ ($3.644 \rightarrow 3.275$) relative to PPOW without R_{dist} . These results support the inclusion of R_{dist} , which preserves performance on easy windows while yielding additional gains on hard windows.

F Additional Baseline Comparisons

F.1 Comparisons with OSD, Lookahead, and FastDraft

Table 9 provides additional comparisons of PPOW against OSD [11], Lookahead [13], and FastDraft [12]. PPOW substantially outperforms OSD on Vicuna-7B [29] and consistently exceeds Lookahead on LLaMA-2-7B [36] over both GSM8K [31] and HumanEval [30]. Relative to FastDraft on LLaMA-3.1-8B-Instruct [27], PPOW achieves markedly larger gains on GSM8K while remaining competitive on HumanEval. These results further confirm the effectiveness of PPOW.

Table 9: **Additional baseline comparisons.** PPOW substantially improves over OSD, Lookahead, and FastDraft.

Target Model	Method	GSM8K		HumanEval	
		τ	Speedup	τ	Speedup
Vicuna-7B	OSD	3.53	1.35×	2.57	1.58×
	PPOW	6.12	4.40×	6.75	4.87×
LLaMA-2-7B	Lookahead	2.26	1.25×	2.42	1.24×
	PPOW	6.50	3.52×	6.68	4.75×
LLaMA-3.1-8B-Instruct	FastDraft	2.67	1.63×	6.99	4.05×
	PPOW	6.50	3.52×	7.23	4.14×

F.2 Additional Results on Natural-Language Tasks

Table 10 reports PPOW results on X-SUM [37] and WMT14 [38]. Compared with the larger improvements observed on GSM8K, the gains on X-SUM and WMT14 are less pronounced. This may be because these tasks are more open-ended, allowing a broader set of continuations to be

Table 10: **Additional natural-language results with LLaMA-3.1-8B-Instruct.** Average acceptance length (τ) and speedup on X-SUM and WMT14 for EAGLE-3, EAGLE-3 (CST), and PPOW. CST denotes continued supervised training from the EAGLE-3 checkpoint.

Method	X-SUM		WMT14		Mean	
	τ	Speedup	τ	Speedup	τ	Speedup
EAGLE-3	4.94	2.81 $\times$	2.72	1.56 $\times$	3.83	2.19 $\times$
EAGLE-3 (CST)	4.58	2.60 $\times$	2.73	1.56 $\times$	3.66	2.08 $\times$
PPOW	5.13	2.87$\times$	2.97	1.64$\times$	4.05	2.26$\times$

accepted during speculative decoding and thereby weakening the acceptance-aware group-relative reward signal used by PPOW. This effect appears weaker on X-SUM because outputs remain partially anchored to the source, and more pronounced on WMT14 because greater lexical and syntactic variation broadens the space of acceptable continuations. This is consistent with our broader claim that PPOW is most effective when speculative performance is governed by relatively structured bottleneck decisions.

G Limitations and Broader Impacts

G.1 Limitations

PPOW improves speculative decoding by aligning training with inference-time acceptance behavior, but this stronger alignment comes with additional training overhead. Compared with supervised drafter training, PPOW requires grouped rollouts, speculative verification, and target-anchored KL regularization, and our current implementation uses both a frozen target model and a trainable drafter. This makes PPOW more resource-intensive and more complex to implement than continued supervised training.

This alignment objective also introduces several hyperparameters beyond standard reinforcement learning, including speculative-decoding-specific settings such as the speculative window size K , the relative cost γ , the proximity reward weight η , the proximity threshold ϵ , and the hard-window curriculum in ADAW. While we use a unified configuration across experiments and observe stable gains, reducing such dependence through more adaptive or self-tuning variants is a natural direction for future work.

G.2 Broader Impacts

PPOW suggests an additional algorithmic perspective on LLM inference: some components of the inference pipeline may be optimized directly with performance-driven objectives rather than only with token-level supervision. In this setting, reinforcement learning provides a practical way to align training with system-level inference behavior when the utility of a model component is determined by structured interactions during decoding rather than by local prediction quality alone.

Framing speculative decoding as a learning problem may also relate to other inference-time decisions, such as candidate allocation, request routing, scheduling, and load balancing. These components are often optimized separately, but in practice they jointly affect end-to-end serving efficiency. The formulation in PPOW may offer a useful perspective for future work that treats such inference-time modules as part of a unified decision-making environment.